From *what happened* to *why it happened*

Causal, span-based tracing of port invocations in F´

nasa/fprime#6227 — and how it relates to #6205, #2784 / #2589 and #1564

Nicolas Rouquette, JPL · prepared for the F´ Change Control Board and community

The claim in one slide

F´ is built on interactions. Components talk only through ports;
requirements that matter at system level are about **chains of port calls**.

F´ cannot observe those chains. Events, telemetry, Autopsy records and
`FW_PORT_TRACING` each say *what* happened at one point. None says *what caused it*.

#6227 asks for one field per port call: its cause. Everything else —
begin/end, arguments, identity — already exists in some form.

#6205 integration-test message log

= spans minus the cause

#2589 / #2784 Autopsy trace records

= spans minus the cause and the end, plus a flight sink

#1564 requirements traceability

= the consumer of the evidence spans give

#6227 every port call is a span, with its cause

framework emits, projects sink, OpenTelemetry data model

Vocabulary (for non-maintainers)

term	meaning here
port call	the only way two F´ components interact; synchronous (runs in the caller's thread) or asynchronous (a message is queued and handled later on the receiver's thread)
EVR / event	a time-tagged record a component chooses to emit; flat, no link to other events
Autopsy	JPL name for a small, always-on, in-memory log of time-tagged trace records, per thread, dumped after a reset (discussion #2589, issue #2784)
span	a unit of work with a begin, an end, attributes, and **a parent or a link to its cause** — the OpenTelemetry data model
trace	the tree (forest) of spans rooted at one external stimulus, e.g. one command
W3C Trace Context	the 25-byte `traceId / spanId / flags` triple that travels with a request so the receiver can attach its span to the sender's
`FW_DIRECT_PORT_CALLS`	F´ build option, on by default outside unit tests: an output port calls the connected handler directly, bypassing the port object

F´'s own documentation sets up the need …

> *"There should be no non-port communication between components."*
> *— user-manual/overview/03-port-comp-top.md*

> *Integration tests make sure "top-level requirements are met" … the tool supports "sending commands, checking for events, and getting telemetry channel readings."*
> *— user-manual/overview/development-practice.md*

> *"system-level issues only appear during integration"*
> *— user-manual/overview/unit-testing.md*

> *The Manager/Worker pattern "typically requires integration testing to ensure that the manager/worker perform in-unison."*
> *— user-manual/design-patterns/manager-worker.md*

Interactions are the architecture; integration is where interaction requirements are checked; **port calls are not observable there.**

… and presents tracing as the remedy, then under-delivers

> *"Events … enable the ability to trace the execution of the system."*
> *— 04-cmd-evt-chn-prm.md — but events are flat, per component, with no causal link*

> *"It is often useful to trace the set of invocations through components and ports."*
> *— configuring-fprime.md, "Port Tracing" — but* `FW_PORT_TRACING` *prints the port name, before the call, with no end, caller, cause or arguments — and is compiled out in default builds (* `FW_DIRECT_PORT_CALLS 1` *)*

- `FpConfig.fpp` declares `FwTraceIdType` ("the type of a trace identifier"); nothing in `Fw/` or `Svc/` uses it
- `RateGroupCycleSlip` reports a cycle number, not which member chain overran
- `REQUIREMENT(...)` exists only in unit tests: of 419 requirements in 59 component SDDs, **304 verify by unit test, 9 by system or integration test**

What vs. *why*

A log is a set

```
{ (t₁, who₁, what₁),
  (t₂, who₂, what₂),
  (t₃, who₃, what₃), … }
```

EVRs, GDS message logs, Autopsy rings. To explain an outcome you must add the
edges yourself, from

1. **the code** — which handler can call which port, and
2. **a timing heuristic** — which of several candidates did.

(1) is engineer time. (2) is wrong exactly under load, priorities and backlog.

A trace is a forest

```
(t₁, who₁, what₁)
└─ (t₂, who₂, what₂)   parent = 1
   └─ (t₃, who₃, what₃) link = 2
```

Same facts, **plus one edge per record**, written at the moment the call is
made, by the code that makes it.

The edge *is* the explanation. No reconstruction, no heuristic. The cost is a
few bytes per record — not the record format, not the sink.

Example: Manager/Worker, requirement MGR-010

`cmdDisp → manager (async workIn) → worker (async workDone) → manager → cmdDisp`; `rateGroup2 → worker.schedIn` (housekeeping). All components active. **MGR-010: every `DO_WORK` completes within 500 ms.** Two `DO_WORK`s arrive within 2 ms; one takes 120 ms, the other 2.1 s.

The event log

```
12.000 cmdDisp OpCodeDispatched DO_WORK seq=41
12.002 cmdDisp OpCodeDispatched DO_WORK seq=42
12.003 manager WorkStarted
12.004 manager WorkStarted
12.110 worker  WorkDone
12.120 cmdDisp OpCodeCompleted  seq=41
…      (hundreds of other lines)
14.102 worker  WorkDone
14.104 cmdDisp OpCodeCompleted  seq=42
```

Shows **that** MGR-010 failed. Which `WorkDone` is seq 42? Queue wait, handler, or return path? Who occupied `worker`?

The trace

```
trace 7f3a cmdDisp.compCmdSend[3] seq=42  12.002→14.104 ✗
└ manager.cmdIn dispatch              12.004→12.005
  └ manager.workOut→worker.workIn enq  12.005
    └ worker.workIn dispatch           14.090→14.101
      ← 2.085 s queued
      └ worker.workDone→manager        14.102→14.103
        └ manager.cmdResponseOut       14.103
trace 91c0 rateGroup2.RateGroupMemberOut[1] 12.006
└ worker.schedIn dispatch             12.110→14.089
  ← 1.979 s handler
```

Shows **why**: seq 42 waited behind a housekeeping handler. Root cause `worker.schedIn`.

What #6227 asks for

1. **An observer interface in `Fw`** (`Fw::PortTraceObserver`) — no I/O, no allocation: `callBegin` / `callEnd` around every output call, `current` (context captured into the queued message), `dispatchBegin` / `dispatchEnd` around the async handler, `dropped` (queue full)
2. **Context propagation** — current span per thread (sync nesting); a fixed-size context serialized into each queued message (async causality, no timing heuristics)
3. **FPP code generation** on every path: `invoke()`, direct-call code, enqueue, `doDispatch()`; optional serialized arguments as attributes
4. **Connection identity in the dictionary** — a `connections` table (id → `source.port[i]`, `dest.port[j]`, graph); today the dictionary has none
5. **Flight constraints** — compile-time switch, off by default, **byte-identical generated code when off**; bounded work; framework does no export

F´ adopts OpenTelemetry's **data model**, not its SDK. No dependency.

The observer events, on the two kinds of call

Synchronous / guarded

caller thread

`callBegin(conn, parent → span)`

handler runs; nested calls become children of *span*

`callEnd(conn, span)`

Parent = the span current on this thread. Nesting is structural.

Asynchronous

caller thread

`callBegin(conn, parent → span)` → `current()` captures *span*, serialized into the message → `callEnd`

or `dropped(conn, cause)` if the queue is full

receiver thread, later

`dispatchBegin(conn, cause = span from message → new span)`

handler runs

`dispatchEnd(conn, span)`

Queue wait = `dispatchBegin` − the enqueueing `callBegin`. Attributed, not inferred.

Why a wrapper outside the generator is not enough

needed	call-site wrapper (post-process generated C++)	#6227
sync begin/end, nesting	yes, if events carry the thread	yes
async causality (which enqueue caused which dispatch)	**no** — the message carries nothing linking it to the sender; order-matching breaks with several senders, priorities, backlog	context travels inside the message
queue wait vs. handler time	no	yes
dropped messages	no — only the receiver's counter increments	`dropped(cause)` names the lost chain
direct port calls (default builds)	only if the topology code is patched too	generated in both modes
destination of each call	no — call site knows its own port, not the peer	dictionary `connections`

The cause has to travel **inside the queued message**, and only generated code builds that message. That is the one line that needs FPP.

Identity: numbers on board, names on the ground

On board, three integers, no strings:

- caller instance number (`getInstance()`, set from FPP's `InstanceIds`)
- FPP-generated per-port id (`PORT_TRACE_ID_*`)
- port index

Each output port element has ≤ 1 connection ⇒ the triple identifies the connection. Optional generated `getConnectionId(inst, port, num)`: a constant-time `switch`, returns the dictionary id.

Off board, the dictionary gains:

```
"connections": [{
  "id": 17,
  "graph": "Commands",
  "source": { "instanceId": 3, "portId": 2,
              "instance": "cmdDisp",
              "port": "compCmdSend", "index": 3 },
  "destination": { … "manager", "cmdIn" … }
}]
```

Spans are labelled with the **model's** names, so a run can be checked against the FPP (or SysML v2) topology mechanically. Ids are build-specific; the dictionary is from the same build.

Four earlier threads, one span

	port calls	enqueue / dispatch	drops	args	begin & end	cause	ids in dictionary	default builds	sink
#6205 integration observability	✓	–	–	✓	–	by matching args	–	–	GDS log
#2589 / #2784 Autopsy, framework half	✓	✓ wait / dequeue	–	✓	–	–	trace ids	–	–
#2589 / #2784 Autopsy, sink half									per-thread ring, DP on reboot, filters
#1564 requirements, verification leg	implied	–	–	–	–	needed	needed	–	–
#6227	✓	✓	✓	optional	✓	**✓ carried**	✓ `connections`	✓	pluggable observer

Each earlier thread is a **projection** of the span — drop the cause, drop the end, fix the sink. None can get the cause from outside the generator.

#6205 — "Full observability of message communication in integration testing"

Havelund, 2026-10: log messages between components **with arguments** to check "A sends M(x) to B, then B within 2 s sends N(x,y) to C"; only unit tests see port histories; maybe a monitor component; worry: timing impact.

#6205	#6227
which messages, between which components	connection id, dictionary names
with arguments	optional argument buffer, reusing port serialization
"then B within 2 s sends N"	N is a **descendant span** of the M dispatch; 2 s is a tree query
in integration, not unit tests	observer runs in the deployed binary, incl. direct-call builds
monitor component	one observer implementation
timing impact	compile-time switch; two observer calls per call; fixed-size context per async message

Narrow case: #6205 correlates M and N through the argument `x` — only works when the developer threaded an id through both interfaces. The carried context works without one, across hops, with shared queues.

#2589 / #2784 — Autopsy: the "what" record, designed well

Tim Canham's concept (#2589) and Shivaly Reddy's implementation (#2784, PR #2953, deferred): automatic trace points for **port call, output port call, message wait, message dequeue**; record = **trace id + time tag + arguments**; `Fw.Trace` port; per-thread in-memory ring, DP on reboot, commands; compile-time **off / minimal / full**; **selectable per component**; user-defined `trace` points.

Framework half → subsumed, + the cause

`TraceCfg.TraceType`	#6227
`PORT_CALL`	`callBegin` / `callEnd`
`MESSAGE_QUEUE`	`callBegin` of the async call (context captured)
`MESSAGE_DEQUEUE`	`dispatchBegin` / `dispatchEnd`
—	`dropped`

A #2784 record is a #6227 span **minus the parent field and the end**.

Sink half → complementary, unchanged

The per-thread ring, recoverable memory, DP on reboot and the enable/filter/dump commands are the **flight-grade** `PortTraceObserver`. #6227 does no buffering or I/O on purpose.

User `trace` **points** attach to the current span automatically — child records of the port call they ran inside.

OS-level categories (interrupts, context switch, semaphores) are outside #6227 but can carry the same context.

#1564 — requirements traceability: the half F´ can own

Out of F´ scope (agreed with thomas-bc's reply): requirements management, baselines, change impact, DOORS/Jama, code annotations, unreferenced-code justification.

The reporter's narrowed question: *how do I ensure the components of a deployment topology meet the upstream requirements?*

Today: `REQUIREMENT(...)` in unit tests only; integration sees commands, events, telemetry — not the interactions the requirements are about.

With connection-keyed spans, an interaction requirement is a predicate over the trace, in the model's ids:

- *for every* `DO_WORK` *span, a* `cmdResponse` *descendant ends within 500 ms*
- *every* `workIn` *dispatch has a* `workDone` *descendant*
- *no span crosses a connection absent from the topology*

Checked mechanically per run; a failure comes with its explaining trace. Works whether the requirement is allocated in FPP, SysML v2, or an external tool referencing topology elements.

Narrow case: #6227 produces the evidence; the bookkeeping stays with the project.

The cause is cheap: 25 bytes is the internet size, not the flight size

W3C context = `traceId` 16 + `spanId` 8 + `flags` 1 = **25 B** per async message and per record — sized for global uniqueness across processes. **On one flight computer, in a per-thread ring:** trace id = root ancestor's id → recoverable offline · sampled flag = deployment decision → 0 bits · sync nesting = record order within a thread (Autopsy already stores per thread) · only **cross-thread edges** (dispatch ← enqueue) need an explicit parent id.

encoding	per async msg	per record
W3C	25	25
compact 64-bit	8	8
parent-pointer 32-bit (`thread:8 ‖ seq:24`)	**4**	4, +4 on dispatch/drop only
parent-pointer 16-bit	2	2 (+2)
fan-in-1 FIFO async port (FPP knows this)	**0**	0 — pairing is a theorem about the topology

`FwTraceIdType` (`U32` , merged for #2784, unused) fits the 32-bit span id. Ground widens injectively: `traceId128 = deployment ‖ boot ‖ 0 ‖ rootSpan` , `spanId64 = boot ‖ span` — still a standard OpenTelemetry trace.

Knobs: compile time, deployment architecture, runtime

Compile time (generated code differs)

- `FW_PORT_SPAN_TRACING` 0/1 — 0 is byte-identical
- level **off / minimal / full** (#2589) — minimal = ids + cause, no args
- context layout: W3C / 64-bit / 32-bit / 16-bit
- per-component / per-port selection (#2589)
- event subset: drop `callEnd` (no durations), drop `dispatchEnd` (implied by next begin)
- time tag: `Fw.Time` 11 B vs. 32-bit monotonic tick + anchor record
- current span in observer table vs. `Os` thread-local storage

Derived by FPP from the topology

- thread index width, per-thread counters (no atomics)
- connection id width (Ref: 68 connections → `U8`)
- sync-only subgraphs: no message context
- fan-in-1 async ports: no message context
- queue depths bound in-flight spans

Runtime (observer; #2784's commands)

- per-connection enable bitset, per-thread enable
- head sampling at roots, inherited by one bit
- kind mask; argument capture and cap
- ring depth; dump trigger (reset, command, monitor verdict)

Implementation status (prototype branches)

FPP — `Open-MBEE/fpp`, branch `feature/port-span-tracing` (on `main`)

- `PORT_TRACE_ID_*` per typed port; `callBegin` / `callEnd` around every output port invocation under `#if FW_PORT_SPAN_TRACING`
- context captured and serialized into async messages; `dropped` on `drop` and `hook` overflow
- topology `getConnectionId()` switch; dictionary `connections` with numeric ids
- existing tests pass; generated C++ compiles with tracing on/off × direct calls on/off

F´ — `Open-MBEE/fprime`, branch `feature/port-span-tracing` (on `devel`)

- `Fw::PortTrace` statics + `Fw::PortTraceObserver`; `FW_PORT_SPAN_TRACING` (default 0)
- unit tests incl. a small topology checking `getConnectionId()` against its dictionary
- two API changes: `getObjName()` const; `getInstance()` public read-only
- doc fixes in "Port Tracing" (`ovrTrace()`, `Fw::Logger::log()`)

Open for the CCB: id scheme; the two API changes; TLS in `Os`?; context layout as a configurable type; adopt #2589's levels.

Proposal to the CCB

1. **Review #6227, #2589 / #2784, #6205 and #1564's verification leg as one topic.**
   #6227 defines the record; #2589 / #2784 provide the on-board sink and user trace points; #6205 is the integration-testing use; #1564 the consumer.
2. **Accept the one new field**: every framework trace record carries its cause, via a context kept per thread and serialized into queued messages.
3. **Make the context layout a configurable `Fw` type** — W3C 25 B for development, 2–4 B parent pointer for Autopsy — and keep #2589's off / minimal / full and per-component knobs.
4. **Let FPP derive widths and context-free connections from the topology**, and emit the `connections` table.
5. Keep **export out of the framework**: observers buffer, filter and ship; the ground maps to OpenTelemetry.

> *Flat logs tell you what happened. The edge tells you why. It costs a few bytes, and only the generator can write it.*

AI disclosure (per `AI_POLICY.md`): drafted with Devin (Cognition AI), reviewed by the author.